

BÀI THỰC HÀNH

XÂY DỰNG BLOG CƠ BẢN VỚI NODE.JS + MONGODB

1. Mục tiêu

- Kết nối Node.js với MongoDB bằng Mongoose
- Tạo model bài viết Blog
- Thêm bài viết mới
- Xem danh sách bài viết
- Xem chi tiết một bài viết
- Hiểu luồng dữ liệu: Form → Server → MongoDB → Giao diện

2. Kiến thức cần lưu ý

- MongoDB không lưu theo bảng như SQL mà lưu theo kiểu:
- Database → Collection → Document
- Ví dụ trong dự án blog:
- Database: blogDB
- Collection: blogposts
- Một document:

```
{  
  "title": "Học MongoDB cơ bản",  
  "body": "Đây là bài viết đầu tiên"  
}
```

3. Nội dung thực hành

Tạo một website blog rất đơn giản gồm 3 chức năng:

- Trang chủ hiển thị danh sách bài viết
- Trang tạo bài viết mới
- Trang chi tiết một bài viết

Ví dụ:

- Người dùng nhập:
 - + Tiêu đề: Học MongoDB cơ bản
 - + Nội dung: MongoDB lưu dữ liệu theo document
- Server nhận dữ liệu
- Lưu vào MongoDB
- Sau đó hiển thị lại trên giao diện

4. Chuẩn bị môi trường

Cài các gói cần thiết. Tạo project Node.js rồi cài thư viện:

```
npm init -y  
npm install express mongoose ejs
```

5. Cấu trúc thư mục

```
project-blog/  
├── app.js  
├── package.json  
├── models/  
│   └── BlogPost.js  
├── views/  
│   ├── index.ejs  
│   ├── create.ejs  
│   └── detail.ejs
```

6. Hướng dẫn thực hành

Bước 1: Tạo Model BlogPost

Tạo file models/BlogPost.js

```
const mongoose = require('mongoose');  
const Schema = mongoose.Schema;  
  
const BlogPostSchema = new Schema({  
  title: String,  
  body: String  
});  
  
const BlogPost = mongoose.model('BlogPost', BlogPostSchema);  
module.exports = BlogPost;
```

Bước 2: Kết nối MongoDB và tạo server

Tạo file app.js

```
const express = require('express');  
const mongoose = require('mongoose');  
const BlogPost = require('./models/BlogPost');  
  
const app = express();  
  
app.use(express.urlencoded({ extended: true }));  
app.set('view engine', 'ejs');
```

```
mongoose.connect('mongodb://127.0.0.1:27017/blogDB')
  .then(() => console.log('Kết nối MongoDB thành công'))
  .catch((error) => console.log('Lỗi kết nối MongoDB:', error));
```

Bước 3: Tạo dữ liệu mẫu và form nhập bài viết

Ví dụ với 1 bài viết có:

- title: Học MongoDB cơ bản
- body: MongoDB là cơ sở dữ liệu NoSQL lưu theo document

Trong tài liệu này, model BlogPost có 2 field là title và body, kết nối bằng mongoose.connect(...), thêm dữ liệu bằng BlogPost.create(...) và lấy dữ liệu bằng BlogPost.find(...).

Tạo file views/create.ejs

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Tạo bài viết</title>
</head>
<body>
  <h1>Tạo bài viết mới</h1>

  <form action="/blogposts/store" method="POST">
    <p>
      Tiêu đề:
      <input type="text" name="title">
    </p>

    <p>
      Nội dung:
      <textarea name="body"></textarea>
    </p>

    <button type="submit">Lưu bài viết</button>
  </form>

  <a href="/">Về trang chủ</a>
</body>
</html>
```

Ý nghĩa: Khi người dùng nhập tiêu đề và nội dung rồi bấm "Lưu bài viết", form sẽ gửi dữ liệu lên server.

3.2. Tạo route mở form

Trong app.js viết:

```
app.get('/blogposts/new', (req, res) => {  
  res.render('create');  
});
```

Ý nghĩa: Khi truy cập vào địa chỉ `http://localhost:3000/blogposts/new` thì trang form nhập bài viết sẽ hiện ra.

3.3. Cho server nhận dữ liệu từ form

Trong app.js phải có dòng này trước:

```
app.use(express.urlencoded({ extended: true }));
```

Dòng này giúp server đọc dữ liệu người dùng gửi từ form HTML. Nếu không có dòng này thì `req.body.title` và `req.body.body` sẽ không lấy được.

3.4. Tạo model BlogPost

Tạo file `models/BlogPost.js`

```
const mongoose = require('mongoose');  
const Schema = mongoose.Schema;  
  
const BlogPostSchema = new Schema({  
  title: String,  
  body: String  
});  
  
const BlogPost = mongoose.model('BlogPost', BlogPostSchema);  
module.exports = BlogPost;
```

Model này được tạo bằng Schema rồi export ra để dùng. Mongoose sẽ tạo collection tương ứng trong MongoDB.

- title
- body

3.5. Kết nối MongoDB

Trong app.js thêm:

```
const mongoose = require('mongoose');  
  
mongoose.connect('mongodb://127.0.0.1:27017/blogDB')  
  .then(() => console.log('Kết nối MongoDB thành công'))  
  .catch((error) => console.log(error));
```

- 127.0.0.1:27017 là địa chỉ MongoDB trên máy
- blogDB là tên database của máy

3.6. Tạo route lưu dữ liệu vào database

Trong app.js thêm:

```
const BlogPost = require('./models/BlogPost');

app.post('/blogposts/store', async (req, res) => {
  console.log(req.body);

  await BlogPost.create({
    title: req.body.title,
    body: req.body.body
  });

  res.redirect('/');
});
```

Khi người dùng nhập form, req.body sẽ có dạng:

```
{
  title: 'Học MongoDB cơ bản',
  body: 'MongoDB là cơ sở dữ liệu NoSQL lưu theo document'
}
```

Và server sẽ lưu vào MongoDB một document như này:

```
{
  "_id": "...",
  "title": "Học MongoDB cơ bản",
  "body": "MongoDB là cơ sở dữ liệu NoSQL lưu theo document"
}
```

3.7. Dữ liệu được lưu trong cơ sở dữ liệu

- Database: blogDB
- Collection: blogposts (Mongoose tự chuyển model BlogPost sang dạng số nhiều)
- Document: chính là từng bài viết

3.8. Lấy dữ liệu từ MongoDB để hiển thị ra trang chủ

Trong app.js viết:

```
app.get('/', async (req, res) => {
  const posts = await BlogPost.find({});
```

```
res.render('index', { posts });
});
```

- BlogPost.find({}) lấy toàn bộ bài viết
- res.render('index', { posts }) gửi dữ liệu qua giao diện

3.9. Hiển thị dữ liệu động ra giao diện

Tạo file views/index.ejs

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Trang chủ</title>
</head>
<body>
  <h1>Danh sách bài viết</h1>
  <a href="/blogposts/new">Tạo bài viết mới</a>

  <ul>
    <% posts.forEach(post => { %>
      <li>
        <strong><%= post.title %></strong><br>
        <span><%= post.body %></span><br>
        <a href="/blogposts/<%= post._id %>">Xem chi tiết</a>
      </li>
      <hr>
    <% }) %>
  </ul>
</body>
</html>
```

Nếu trong database đã có bài viết thì giao diện sẽ hiển thị tiêu đề, nội dung và link xem chi tiết.

3.10. Xem chi tiết một bài viết

Tạo route chi tiết trong app.js:

```
app.get('/blogposts/:id', async (req, res) => {
  const post = await BlogPost.findById(req.params.id);
  res.render('detail', { post });
});
```

Tạo file views/detail.ejs

```
<!DOCTYPE html>
<html>
```

```
<head>
  <meta charset="UTF-8">
  <title>Chi tiết bài viết</title>
</head>
<body>
  <h1><%= post.title %></h1>
  <p><%= post.body %></p>

  <a href="/">Về trang chủ</a>
</body>
</html>
```

Ý nghĩa: Khi người dùng bấm "Xem chi tiết", server sẽ lấy bài viết theo `_id` rồi hiển thị nội dung đầy đủ.

7. Luồng chạy thực tế cho bài thực hành

- Bước 1: Mở trình duyệt vào: `http://localhost:3000/blogposts/new`
- Bước 2: Nhập tiêu đề và nội dung bài viết
- Bước 3: Bấm nút Lưu bài viết
- Bước 4: Form gửi dữ liệu đến route: `POST /blogposts/store`
- Bước 5: Server nhận được: `req.body.title` và `req.body.body`
- Bước 6: Server chạy `BlogPost.create(...)` để lưu vào MongoDB
- Bước 7: Sau khi lưu xong, server chuyển về trang chủ bằng `res.redirect('/')`
- Bước 8: Trang chủ chạy `BlogPost.find({})` để lấy tất cả bài viết ra
- Bước 9: EJS hiển thị dữ liệu động lên web
- Bước 10: Khi bấm "Xem chi tiết", trình duyệt gọi `/blogposts/:id` để hiển thị 1 bài viết cụ thể

8. Code mẫu hoàn chỉnh cho bài thực hành

File `app.js`

```
const express = require('express');
const mongoose = require('mongoose');
const BlogPost = require('./models/BlogPost');

const app = express();

app.use(express.urlencoded({ extended: true }));
app.set('view engine', 'ejs');

mongoose.connect('mongodb://127.0.0.1:27017/blogDB')
  .then(() => console.log('Kết nối MongoDB thành công'))
  .catch((error) => console.log(error));
```

```
app.get('/', async (req, res) => {
  const posts = await BlogPost.find({});
  res.render('index', { posts });
});

app.get('/blogposts/new', (req, res) => {
  res.render('create');
});

app.post('/blogposts/store', async (req, res) => {
  await BlogPost.create({
    title: req.body.title,
    body: req.body.body
  });

  res.redirect('/');
});

app.get('/blogposts/:id', async (req, res) => {
  const post = await BlogPost.findById(req.params.id);
  res.render('detail', { post });
});

app.listen(3000, () => {
  console.log('Server đang chạy tại http://localhost:3000');
});
```

File models/BlogPost.js

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const BlogPostSchema = new Schema({
  title: String,
  body: String
});

const BlogPost = mongoose.model('BlogPost', BlogPostSchema);
module.exports = BlogPost;
```

File views/create.ejs

```
<!DOCTYPE html>
<html>
<head>
```



```

<meta charset="UTF-8">
<title>Tạo bài viết</title>
</head>
<body>
  <h1>Tạo bài viết mới</h1>

  <form action="/blogposts/store" method="POST">
    <p>
      Tiêu đề:
      <input type="text" name="title">
    </p>

    <p>
      Nội dung:
      <textarea name="body"></textarea>
    </p>

    <button type="submit">Lưu bài viết</button>
  </form>

  <a href="/">Về trang chủ</a>
</body>
</html>

```

File views/index.ejs

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Trang chủ</title>
</head>
<body>
  <h1>Danh sách bài viết</h1>
  <a href="/blogposts/new">Tạo bài viết mới</a>

  <ul>
    <% posts.forEach(post => { %>
      <li>
        <strong><%= post.title %></strong><br>
        <span><%= post.body %></span><br>
        <a href="/blogposts/<%= post._id %>">Xem chi tiết</a>
      </li>
      <hr>
    <% }) %>
  </ul>

```

```
</body>
</html>
```

File views/detail.ejs

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Chi tiết bài viết</title>
</head>
<body>
  <h1><%= post.title %></h1>
  <p><%= post.body %></p>
  <a href="/">Về trang chủ</a>
</body>
</html>
```

9. Chạy chương trình, kiểm tra kết quả và nộp bài

9.1. Sinh viên test kết bằng lệnh node app.js và thử nghiệm

9.2. Nộp bài:

Ảnh chụp màn hình kết quả chạy chương trình

Sinh viên phải chụp màn hình kết quả chạy của website trên trình duyệt tại các địa chỉ sau:

Trang chủ hiển thị danh sách bài viết: <http://localhost:3000>

Trang tạo bài viết mới: <http://localhost:3000/blogposts/new>

10. Phần làm thêm

10.1. Thêm CSS

10.2. Sắp xếp bài mới lên trên

10.3. Thêm chức năng sửa / xóa bài viết